

The Total-Work Cost of Bounded Retry Guarantees

ANONYMOUS AUTHOR(S)

Optimistic completion trades unprotected preparation against recomputation under a guard. Bounding calls alone does not bound duplicated work. For persistent whole-kernel jobs, we characterize the separate worst total-work and protected-work guarantees, including the cost of one policy that must provide optimal protection at every hidden write budget. Independent jobs admit complete polynomial frontiers and an exact least total-work curve. A domination theorem reduces all admitted programs, including early preparation and retained records, to finite immediate-preparation trees. Observation erasure, changed call units and guard-entry charges expose boundaries of these results; unit guards make joint caps strongly NP-complete on height-two DAGs despite polynomial output size. We prove a conditional transfer to per-key Java wrappers. A native Linux pathname variant illustrates the resource choice: reducing protected rebuilding after one write permits a third body attempt instead of upstream's maximum two. Roslyn preservation and mixed timing results delimit broader transfer. These are operation/resource guarantees, not measured latency guarantees.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, scheduling, online policies

1 Introduction

After validation fails, a policy can retry unprotected, recompute under protection, or prepare for guarded reuse/recomputation. These choices can share a call cap but differ in total/protected work. Which joint guarantees are attainable when the write budget is supplied or unknown?

The choice occurs in actual fallback code. Linux's conditional-reader idiom tries locklessly before acquiring a lock for progress [11]. In the single rename-lock pathname loop, upstream performs at most two body attempts. Our candidate allows a second outside preparation before guarded reuse/rebuilding: it avoids protected body work after one write, but may perform a third body attempt after two. It therefore relaxes a source-derived body-attempt bound even though both logical call caps are two. Section 8.2 separates that control-flow fact from primitive waiting, native time and any caller-imposed requirement.

The issue also depends on information. With independent works 8 and 16, supplied write bound $B = 1$ and two calls, cache the 8-unit job; on match finish the other fresh, and on mismatch prepare/publish it cheaply. The paths cost $(W, L) = (24, 16)$ and $(32, 8)$, so the separate worst-case pair is $(32, 16)$. All-fresh gives $(24, 24)$. One policy providing optimal protection at every hidden budget instead requires $(40, 16)$ at $B = 1$. Separate maxima need not occur on the same execution.

We give polynomial independent-job frontiers, a least unaware total-work curve, and finite-tree domination covering free inspections and retained preparations. Observation/accounting boundaries include unit-guard precedence hardness. Java provides a conditional realization; Linux and Roslyn expose transfer limits. Fallback, testing and minimax dynamic programming are inherited; Section 9 distinguishes our structural results.

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$, $n \geq 1$, form a finite DAG with predecessor set $\text{Pred}(i)$. Readiness requires all predecessors completed. Completion records an exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction. Policies are deterministic and causal; no randomized expected-cost guarantee is claimed.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Every execution performs its fixed work $w_i > 0$, whether inside or outside protection. Partial/incremental evaluation, preparation before readiness and discharge by external work are excluded in this interface. A preparation is tied to its own-input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

States, histories and turns. A state $(x, \mathcal{M}, \mathcal{R}, \ell)$ contains live own-input identities, completed immutable outputs, a finite snapshot/record store, and program locals. A record carries its job, captured identity, predecessor tuple and computed result. Records may be retained, copied and selected, but not fabricated or retagged without computation. In Table 2, U means no selected record, C a selected possibly stale record, and D permanent completion. A cheap failure clears selection but preserves retained copies. At every finite history each unfinished job admits a legally writable identity distinct from every target identity in its history, records and raw snapshots; the adversary may choose never to reuse it. This includes old snapshots from before the charged interval. Finiteness alone does not imply such a supply. Writes preserve completed outputs.

Visible histories list actions, returned values and completion events. Cached completion exposes its identity-comparison Boolean alongside the completed output; pure recomputation may return the same canonical result. Completion publishes its saved output over the live own-key entry. A program chooses a ready job/mode or an unprotected inspection, preparation or local action. The environment then selects a finite word of legal writes: the operation's *gate*, before its indivisible protected part for a callback. Job/mode is fixed before the gate; later record selection may use only the already-computed prior store. Even a paid new preparation cannot be used after this gate in the same selected call: it is a separate action with a later comparison gate. A cached mismatch recomputes and completes within the same counted call; its exact comparison has no spurious mismatch. Completing comparison/computation/publication is indivisible to writes, and no guard survives return. Inspection honestly returns the live entry.

Immediate, or just-in-time, preparation captures and computes only for the next selected call. General programs may prepare earlier after readiness, from immutable own snapshots and exact completed parents; old raw snapshots may predate the counted writer interval. Its pure body has no live read or interaction. At completed-operation boundaries, contract it with its returned record and full work, replaying writes since capture before the next operation. Section 6's relation retains pending writes and does not place a result or full charge at an unfinished concrete prefix.

The common start precedes all computed initialization: every kernel execution contributes to W , including discarded preparations and computation from old raw snapshots. No fixed initialization charge is mandatory. L counts its protected part. Q counts validation/publication/completion calls, including failures; preparations and writers do not consume it. Waiting, common call overhead and nonkernel allocation are excluded. Neither clocks nor work counters are implicitly observed. Write $\Omega_k(T)$ for the sum of the largest $\min(k, |T|)$ works and $\Omega(T)$ for the total; omitted sets mean J , and $\Omega_0 = 0$. These aggregate/allocation objects are inherited [16, 17].

A cheap call publishes only on match and exposes success/failure. Conditional publication and a validation-only callback discarding its raw result implement this transition (Section 6). Two modes offer cheap/fresh operations; three add cached completion. The label "cheap" concerns kernel work. Failed validation leaves the entry unchanged. Free inspections/local record operations neither publish nor retain a guard; only a publishing completion can change an unfinished job's live identity through a foreground action. Direct acquire/release, retained protection, cross-job callbacks and outside completion are excluded, so this is not an arbitrary-lock-API lower bound. Selected operations must terminate. Admissibility requires all jobs to complete after finitely many

Table 1. Resources and information contracts used in the results.

Symbol	Meaning
$Q \leq n + r$	At most r noncompleting calls beyond the n mandatory completions.
B	Upper bound on external writes; an input only for informed policies.
$a_1 \leq \dots \leq a_n, A_k$	Sorted works and sum of the k smallest works, $A_0 = 0$.
$\mathcal{P}_3(B, r)$	Policies requiring the call cap only against at most B writes.
$\mathcal{U}_3(r)$	One budget-unaware policy requiring the cap at every finite budget.
$\mathcal{V}_r$	Policies in $\mathcal{U}_3(r)$ also attaining optimal L at every budget.

Table 2. Whole-kernel operations. Every validation or callback adds one to Q ; preparation adds none. Entries show successor state and added (W, L) . Each callback holds protection only within that operation.

Operation	Start	Match	Mismatch
Prepare	U	C, $(w_i, 0)$	—
Cheap publication	C	D, $(0, 0)$	U, $(0, 0)$
Fresh callback	U	D, (w_i, w_i)	—
Cached callback	C	D, $(0, 0)$	D, (w_i, w_i)

actions on every admitted play; endless free actions are not completion. This does not impose a uniform action bound across environments.

Strategies and admissibility. An informed strategy is a deterministic function $P(B, h)$; an unaware strategy is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes along every play. For mode family j , $\mathcal{P}_j(B, r)$ consists of strategies completing with $Q \leq n + r$ against every $E \in \mathcal{E}_B$. By contrast, $\mathcal{U}_j(r)$ consists of one B -independent strategy satisfying that cap for every $b \geq 0$ and $E \in \mathcal{E}_b$. The adaptive gate adversary knows the deterministic program and its prior store; it acts after selection and before execution, even after a free inspection. A fixed deterministic adverse play can be scripted at gates; this does not establish a wall-clock-oblivious or randomized value. Every finite-write play against a fixed program is preserved by capping its environment at that play’s write count. Worst cost at budget B is $\sup_{E \in \mathcal{E}_B} L(P, E)$; the two minima use different admissible classes.

LEMMA 1 (FRESH-WRITE AND FINITE-ADVERSARY FACTS). *In Section 2’s game, a fresh own-key write at a selected i -operation’s gate invalidates every retained i -record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment’s budget. An adversary bounded by b writes on every play forces any $P \in \mathcal{P}_j(b, r)$ to finish every job, regardless of free inspections or record retention.*

PROOF. A fresh identity differs from every retained target identity. Copying/selection cannot change that, and all observations and job/mode choices agree until the gate. The bounded adversary belongs to $\mathcal{E}_b$; leaving any job unfinished violates admissibility even with infinitely many free actions. Thus every job completes through Table 2. No unbounded adversary is truncated. $\square$

LEMMA 2 (WORK CHARGED BY USEFUL COMPARISON WRITES). *From the charged common start, consider a complete execution in which the adversary writes only fresh own-key identities at comparisons whose zero-write continuation would match. Let H be the multiset of resulting cheap failures and D*

the set of resulting cached mismatches. Then

$$W \geq \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

Already-stale failures may consume calls, but need no extra charge in this inequality.

PROOF. At each useful comparison, charge the record that the no-write outcome of *that same* call would select, even if its failure branch selects an older one. The creating calculation is in the pre-gate store and is therefore paid on both branches. The fresh write invalidates every prior target record. Before completion, neither failed cheap calls nor other foreground actions restore an earlier live identity; the restricted adversary never restores one. Each later useful comparison therefore charges a different computed identity. Final completion needs a further distinct execution: protected work or a matching preparation for the final identity. A cached mismatch pays both its pre-gate preparation and recomputation. Different jobs cannot share charges. These calculation instances are pairwise distinct across useful failures, cached mismatches and mandatory completions. A stale cached call supplies its mandatory protected calculation; its old preparation is unused slack in the bound. This handles old snapshots and early computation globally; a suffix may have prepaid records. $\square$

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For two modes when $B > r$, reject cheap calls with fresh writes, stopping unconditionally after $r + 1 \leq B$ rejections. The call cap forbids exhausting this budget, so every job completes under protection.

For three modes, target the $k = \min(B - r, n) > 0$ heaviest jobs. Count noncompleting target comparisons f and completed targets p , initially zero. Write a fresh identity only at target comparison gates while $f \leq r$ and $p < k$; stop permanently at $f = r + 1$ or $p = k$. Counters follow actual outcomes, including fresh completions. While active, cheap target calls fail, cached calls recompute, and fresh completion protects its body without a write.

At operation boundaries the write count satisfies $v \leq f + p$: each write caused a failure or cached completion. Before another write, $v + 1 \leq f + p + 1 \leq r + k \leq B$. This is unconditionally an $\mathcal{E}_B$ strategy, even on plays violating the call cap.

Admissibility excludes $f = r + 1$: those failures plus n mandatory completions violate the cap. Lemma 1 forces every target to complete, fresh or by cached mismatch, protecting its body. Hence $p = k$ and $L \geq \Omega_k$. Free inspections, stale selections and retained copies cannot evade this. Predecessors contribute their own mandatory calls; target predecessors are counted once in p . $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the protection ratio is $n/\min(B - r, n)$. At $B = r$, informed two-mode policies know no write remains; unaware policies must tolerate another-write extensions. Per-update and shared-batch call allowances are distinct; the Roslyn comparison reports both caps.

4 The Total Work Needed for Optimal Protection

The protection optimum alone permits wasteful preparation. We minimize total work under simultaneous and supplied-budget protection contracts, for arbitrary r . The separate suprema $(\sup W, \sup L)$ may have different witnesses; they do not determine $\sup(W + \lambda L)$. All computed initialization remains charged.

Two bodies and two calls are different contracts. For one unit-work job, a single program receiving no B and completing with at most two body executions at every finite write budget must have worst protected body work at least one already at $B = 1$. Suppose instead it never protects a body under one write. Its first no-write-matching comparison must be cheap: a fresh gate write would force a cached body. Force that cheap failure, then stop writing. Finite completion without protection requires another matching prepared record. In a second environment, invalidate this record at its comparison too. Lemma 2 charges two distinct invalidated preparations and a mandatory final body, contradicting the global two-body cap. Both environments have unconditional one/two-write bounds. No call cap is used in this argument.

Cheap followed by fresh attains the two-body cap. Repreparing for cached completion instead gives worst (W, L) pairs $(1, 0)$, $(2, 0)$ and $(3, 1)$ at budgets 0, 1 and $B \geq 2$, still with $Q \leq 2$. The whole-kernel premise excludes a native lower bound for partial snapshots or variable body work.

4.1 One Policy with Optimal Curves at Every Budget

Let $\mathcal{V}_r$ contain the policies in $\mathcal{U}_3(r)$ satisfying $L(P, E) \leq \Omega_{(b-r)_+}$ for every $b \geq 0$ and $E \in \mathcal{E}_b$. This requires one *simultaneous optimal-protection curve*; it is stronger than universal completion. Write $W_P(B) = \sup_{E \in \mathcal{E}_B} W(P, E)$.

LEMMA 3 (CAPPING A REALIZED PLAY). *For $P \in \mathcal{V}_r$, a finite prefix containing d actual writes has protected work at most $\Omega_{(d-r)_+}$.*

PROOF. Continue the prefix without writes. Admissibility forces completion. Cap that environment unconditionally at d : induction preserves each action and realized write, and there are no

later writes to remove. The resulting environment belongs to $\mathcal{E}_d$ and preserves the complete play. Its total protected work satisfies the bound; nonnegative costs give the prefix bound. The program need not observe d . $\square$

THEOREM 3 (LEAST TOTAL-WORK CURVE UNDER SIMULTANEOUS OPTIMAL PROTECTION). *For independent jobs with sorted positive works $a_1 \leq \dots \leq a_n$,*

$$\inf_{P \in \mathcal{V}_r} W_P(B) = F_r(B) := \begin{cases} \Omega + Ba_n, & B \leq r, \\ \Omega + \max_{1 \leq m \leq \min(B-r, n)} \{ra_{n-m+1} + \Omega_m\}, & B > r. \end{cases}$$

One policy attains this entire curve: process jobs in ascending work order, freshly prepare before each cheap call until r cheap failures, and freshly prepare before each cached completion thereafter. It observes neither B nor cached comparison results. Sorting, prefix sums and prefix maxima construct the curve in $O(n \log n)$ arithmetic/comparison operations, representing its initial linear segment analytically.

PROOF. *Upper bound.* Each bad comparison needs a write in its distinct preparation-to-comparison interval. There are at most r cheap failures; cached mismatches occur only thereafter, at distinct jobs. Thus $Q \leq n + r$ and $L \leq \Omega_{(B-r)_+}$ at every budget. When $B \leq r$, at most B preparations are lost, each costing at most a_n .

For $B > r$, suppose cached mode starts at rank j and has $m \geq 1$ mismatches. All failed preparations were at ranks at most j and the m mismatched jobs lie in the suffix, so $j \leq n - m + 1$. Duplicate work is at most $ra_j + \Omega_m \leq ra_{n-m+1} + \Omega_m$, with $m \leq B - r$. Without a mismatch it is at most ra_n , below the $m = 1$ term. For each permitted m , match the first $n - m$ jobs, fail cheap r times at the next job, and mismatch all m cached completions. This attains its term using $r + m$ writes. The analogous maximum-job witness attains $\Omega + Ba_n$ at $B \leq r$.

Lower bound for arbitrary programs. Fix $m \leq \min(B - r, n)$ and target the m heaviest jobs. Write a fresh own-key identity only at target comparisons whose zero-write continuation would match. Let f count these useful cheap failures, c useful cached mismatches and t all target completions. Stop forever after $f = r + 1$ or $t = m$. At operation boundaries the write count is $d = f + c$, with $c \leq t$. Before another write, $d + 1 \leq f + t + 1 \leq r + m$. This is a globally $(r + m)$ -bounded adversary even on inadmissible programs.

An admissible program cannot reach $f = r + 1$, and every target eventually completes. Before all targets finish, no target comparison can match: a useful comparison is invalidated, and a stale one already fails or protects. Thus all targets complete protectively and $L \geq \Omega_m$. Lemma 3, applied at the realized final count d , gives

$$\Omega_m \leq L \leq \Omega_{(d-r)_+}, \quad d = f + c \leq r + m.$$

Positivity forces $d = r + m$, hence $f = r$ and $c = m$. All target completions are useful cached mismatches; this is a consequence of the contract, not an assumed normal form. Lemma 2 now charges r distinct failed preparations of work at least a_{n-m+1} , all m duplicate target kernels and mandatory Ω . Maximizing over m gives the lower bound.

Early budgets. The $m = 1$ witness just constructed has r useful cheap failures at a maximum-work job, then its useful cached mismatch. Truncate after the first $B \leq r$ writes. Its unchanged prefix has destroyed B distinct paid preparations of that unfinished job. Eventual completion needs another distinct kernel, giving $\Omega + Ba_n$ by global charging. At $B = 0$ mandatory work suffices. Lower bounds include retained records, free inspections and paid early preparations after readiness. $\square$

The maximizing m need not be largest: works $(1, 100)$ and $r = 1$ give terms 200 and 102. Optimal W and optimal L may have different maximizing executions. The theorem identifies one least

attainable pair of curves, not the work of every optimal- L policy. A relaxed protection requirement defines a different comparison.

Dependency boundary. The lower bound is order-independent. For any DAG, any topological cheap-then-cached policy attains $F_r(B) = \Omega + Bw_{\max}$ at $B \leq r + 1$, while retaining optimal L at all budgets. At $r = 0$, all-cached gives $F_0(B) = \Omega + \Omega_B$ for every DAG and budget. Later positive-retry values can differ: the chain $10 \rightarrow 1 \rightarrow 1$, $r = 1$, $B = 3$, requires $W = 33$, versus 32 for independent jobs. Two target writes force a cheap failure and cached mismatch at the 10-unit job. Its protected work already equals $\Omega_1 = 10$; the next job cannot be fresh on the no-more-write continuation, nor cheap with no retry left. A third write duplicates that unit: mandatory work 12, a failed 10-unit preparation, a cached 10-unit duplicate and one unit give $12 + 10 + 10 + 1 = 33$. The chain policy attains this value. A least entire curve for arbitrary positive-retry DAGs remains unresolved.

4.2 The Complete Frontier for Independent Jobs

A supplied write bound permits a different policy for each resource contract. Here allow arbitrary retry slack $r \geq 0$. Sort independent jobs by positive work $a_1 \leq \dots \leq a_n$ and let $A_k = \sum_{j=1}^k a_j$, $A_0 = 0$. Put $h = \min(r, B)$ and $p = B - h = (B - r)_+$.

THEOREM 4 (INDEPENDENT-JOB FRONTIER). *For independent jobs, the exact $(\sup W, \sup L)$ frontier over $\mathcal{P}_3(B, r)$ is the nondominated subset of (Ω, Ω) and*

$$(\Omega + C_k, \Omega - A_k), \quad 1 \leq k \leq n - p,$$

$$C_k = ha_k + A_{k+p-1} - A_{k-1}.$$

For $p = 0$ the prefix difference is zero. If $p \geq n$ only (Ω, Ω) remains; at $B = 0$ the frontier is $(\Omega, 0)$. Sorting and a prefix-sum sweep construct the frontier in $O(n \log n)$ arithmetic/comparison operations.

PROOF. For selected k , process jobs in ascending work order. Count matching completions s and cheap failures f . Before each preparation, if $s = k$, finish the suffix fresh; otherwise prepare/cheap when $f < r$ and prepare/cached when $f = r$. Stop when all jobs finish. This policy reads no B after k is selected.

Let m count cached mismatches. Disjoint invalidation intervals give $f + m \leq B$; cached mode requires $f = r$, so $m \leq p$. If all jobs finished before $s = k$, then $n = s + m \leq k - 1 + p < n$, a contradiction. Hence exactly k matching jobs precede the fresh suffix. Their weight is at least A_k , giving $L \leq \Omega - A_k$, attained without writes.

Cheap failures occur before cached mode, at rank $s + 1 \leq k$, and cost at most ha_k . The j th cached mismatch has rank $s + j \leq k + j - 1$. Thus $W - \Omega \leq ha_k + \sum_{j=k}^{k+p-1} a_j = C_k$. To attain it, match the first $k - 1$ jobs, fail cheap h times on job k , then cause p cached mismatches at ranks $k, \dots, k + p - 1$. The next job $k + p \leq n$ matches and triggers fresh completion. For $p = 0$, job k itself matches after the failures. The call bound is $Q \leq n + h$ within budget and $Q \leq n + r$ against any finite number of writes.

For the lower bound, target ranks $k, \dots, n$. A globally B -bounded adversary writes only at target comparisons whose zero-write continuation would match. Already-stale cheap calls receive no write; stale cached calls protect without one. If the budget is not exhausted, every target completes protectively, giving $L \geq \Omega - A_{k-1}$. Otherwise let $f \leq h$ writes cause cheap failures; the other $B - f$ cause cached mismatches at distinct targets. Lemma 2 gives

$$W - \Omega \geq fa_k + \sum_{j=k}^{k+B-f-1} a_j \geq C_k.$$

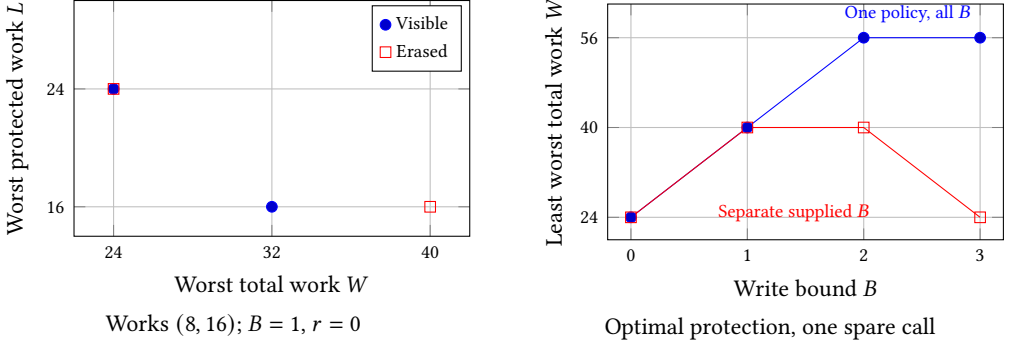


Fig. 1. Independent works (8, 16). Left: exact frontier points when the cached comparison is visible or fully erased (Section 5.2); no interpolation. Right: one unaware policy or separate supplied policies attain $L^*(B) = (0, 0, 16, 24)$ for $B = 0, \dots, 3$. The supplied total optimum can fall as its protection cap loosens.

The actual distinct targets ensure this window exists. Increasing f replaces its last weight by a_k and cannot increase it; at $f = h$ it equals C_k . Wasted stale failures consume retry allowance without reducing this bound.

Consequently worst $L < \Omega - A_{k-1}$ forces worst $W \geq \Omega + C_k$. Since C_k is nondecreasing, a cap $M < C_1$ forces all work protected. Otherwise choose the largest k with $C_k \leq M$; the next index's target bound forces $L \geq \Omega - A_k$. At $k = n - p$, Theorem 1 supplies the same lower bound Ω_p . The attaining policies complete the frontier; nondominance removes tied thresholds. At $B = 0$, select $k = n$; at $p \geq n$, mandatory work and the protected-work lower bound give all-fresh. $\square$

Selecting k uses the supplied contract. Its constant protection cap holds exactly through $B = r + n - k$; the next budget permits r failures and $n - k + 1$ cached mismatches.

4.3 When Retry Slack Covers the Write Budget

PROPOSITION 5 (AN ORDER-INDEPENDENT FRONTIER WITHOUT THE CACHED FLAG). *If $r \geq B > 0$, on every DAG the least worst L under $W \leq \Omega + M$ is*

$$\sum_{i: Bw_i > M} w_i.$$

Its attaining policy needs only cheap success/failure, even under the output-only cached contract of Section 5.2.

PROOF. Choose the k smallest jobs for cheap calls and all others fresh; follow any topological order until B cheap failures, then prepare/cache every unfinished job. Duplicate work is at most Ba_k , and protected work at most $\Omega - A_k$. Repeated writes at a maximum chosen job and the no-write path attain the separate bounds. Theorem 4's target lower bound uses no execution order and specializes to $C_k = Ba_k$. Tied thresholds give the displayed cap form. $\square$

4.4 If a Cached Mismatch Consumes Another Call Unit

A logical completion call need not equal a consumer's resource unit. Keep the same atomic operations and observations, but charge a cached mismatch two units; every other cheap/fresh/cached call costs one. Let Q' count these units. At a boundary with u completions and z cheap failures plus cached mismatches, $Q' = u + z$; at completion it is $n + z$. This changes accounting, without splitting the callback into interleavable operations.

PROPOSITION 6 (DEPENDENCE ON THE CALL UNIT). *Under $Q' \leq n + r$, the supplied-budget body frontier on every DAG is $\{(\Omega, \Omega)\}$ when $B > r$, $\{(\Omega, 0)\}$ when $B = 0$, and Proposition 5's frontier when $0 < B \leq r$. Among policies admissible at every finite budget, one policy attains the least protected-work curve 0 for $B < r$ and Ω for $B \geq r$.*

PROOF. For $B > r$, write a fresh own-key identity at every cheap/cached gate whose no-write continuation would match. Stop permanently at $z = r + 1$. At boundaries $v \leq z$, so each useful write while $z \leq r$ respects $v + 1 \leq r + 1 \leq B$, even on inadmissible histories. Every cheap call fails and every cached completion mismatches while active. Admissibility forbids $z = r + 1$; finite completion therefore protects every body. All-fresh attains (Ω, Ω) .

For $B \leq r$, $Q' \geq Q$ inherits the original lower bound. Proposition 5's attaining policy has at most B cheap failures and only matching cached calls after budget exhaustion, so $Q' = Q$. All cheap gives the $B = 0$ pair.

For the unaware upper bound, use cheap calls with immediate preparation until r failures, then all-fresh. For its lower bound at $B \geq r$, use the same reject-all-comparisons adversary but stop at $z = r$, using at most r actual writes. Every earlier completion is protected. If unfinished jobs remain, any further cheap/cached call is already stale or could become bad after one more write in another finite environment, violating universal admissibility. That hypothetical write is absent from the realized play. Thus all remaining completions are fresh. If every job completed earlier the bound already holds. Lemma 1 handles retained records, free inspections and mandatory finite completion. $\square$

Section 8.2 compares logical completion blocks with native call counts.

5 A Complete Finite Policy Class for Arbitrary Retry Slack

We now justify finite policy search for general DAG/budget combinations against all admitted programs.

THEOREM 7 (RESOURCE DOMINATION BY IMMEDIATE-PREPARATION TREES). *For every $P \in \mathcal{P}_3(B, r)$ there is a finite fresh/cheap/cached tree T such that*

$$\forall E \in \mathcal{E}_B \exists E' \in \mathcal{E}_B : (Q, W, L)(T, E) \preceq (Q, W, L)(P, E').$$

Its depth is at most $n + r$, and it prepares only immediately before its selected cheap/cached node.

PROOF. Fix P , the instance and supplied B . We construct a tree with a *concrete original history* attached to every node; scheduling by a completed set alone is not assumed sufficient.

Extract the witnesses. Give no writes to inspections, preparations and local actions. At a selected comparison, inspect the deterministic no-write outcome of *that call*. If it matches, retain no-write and one fresh-write branches, fixing a legal record-invalidating identity at this history. Cheap gives completing/failing children; cached gives two completing children. Fresh gives one completing child. An already-stale cached call becomes a fresh node with its original no-write continuation. Delete an already-stale cheap call and retain its failing continuation. All intervening local choices, values and records remain in the concrete witness, not the emitted tree. Stop this extraction when B useful writes have occurred.

Finite structure. If an original prefix has u completions, h useful cheap failures and s deleted stale failures, it has $u + h + s$ calls; the projection has $u + h$. Every such prefix has a no-more-write extension in $\mathcal{E}_B$. Admissibility excludes excess calls and infinite intervening actions. Thus retained depth is at most $n + r$; finite branching gives a finite tree. Projected completions preserve readiness, and deleted calls complete no job.

Global charges and the cut tail. Complete each original cut witness by running P with no more writes. On a complete witness, let H be retained cheap failures and D retained cached mismatches. Lemma 2 reserves one distinct mandatory calculation per job and one per element of H, D :

$$W_P \geq \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

Each projected fresh node corresponds to an original protected completion, including a collapsed stale cached call. Together with D these give distinct protected calculations and the analogous lower bound on L_P .

Replace the tree's cut continuation by immediate prepare/cached completions in a fixed ready order. In any actual execution, the preceding B bad comparisons require B distinct writes, so the suffix matches throughout. It needs exactly $n - u$ calls and zero protected work. Original P needs at least $n - u$ further completing calls. The replacement's suffix preparations are covered by the *global* mandatory charges: even an original suffix completion using an early record has its distinct calculation somewhere in the charged execution. We do not assert that its suffix alone owes $\Omega(S)$. For $B = 0$, this applies at the root.

Realize one operational tree. At each node use the labeled ready job/mode; immediately capture and prepare for cheap/cached, and do no preparation for fresh. Follow the actual returned branch bit, ignoring concrete values for scheduling. Its correct saved outputs follow from the interface, although its values need not equal P 's witness values. Each bad comparison needs a write in its fresh preparation-to-comparison interval. These intervals are disjoint, so an actual path in $\mathcal{E}_B$ has $d = |H| + |D| \leq B$. Its exact costs are

$$Q_T = n + |H|, \quad W_T = \Omega + \sum_H w_i + \sum_D w_i, \quad L_T = \sum_{\text{fresh}} w_i + \sum_D w_i.$$

Match each actual path. Follow its corresponding concrete witness of P , choosing the fixed fresh write exactly at its d bad binary nodes, and the retained no-write continuations elsewhere. After a cut, use the original no-write suffix. Define E' to make these writes only on the prescribed witness prefixes, stop on every other history, and impose an unconditional cap d . Thus $E' \in \mathcal{E}_d \subseteq \mathcal{E}_B$ on every play. Determinism reproduces this witness. Its calls number at least $n + |H|$, and the global bounds dominate the displayed W_T, L_T . This proves the stated quantifiers; different real paths may use different E' , with no equality under the same environment asserted. $\square$

5.1 Exact Pareto Recurrence

Let $\mathcal{F}(S, b, a)$ contain nondominated suffix-tree bounds (e, l) , where S is unfinished, b is a conservative remaining write allowance, a remaining noncompletions, $e = \max W - \Omega(S)$ and $l = \max L$. Set

$$\mathcal{F}(\emptyset, b, a) = \mathcal{F}(S, 0, a) = \{(0, 0)\}.$$

For ready i , write $R = S \setminus \{i\}$ and take the nondominated union of

$$\begin{aligned} \text{fresh:} & \quad (e, w_i + l), \quad (e, l) \in \mathcal{F}(R, b, a); \\ \text{cached:} & \quad (\max\{e_0, w_i + e_1\}, \max\{l_0, w_i + l_1\}); \\ \text{cheap, } a > 0 : & \quad (\max\{e_0, w_i + e_1\}, \max\{l_0, l_1\}). \end{aligned}$$

Both comparisons choose their matching child in $\mathcal{F}(R, b, a)$. Cached chooses its other child in $\mathcal{F}(R, b - 1, a)$; cheap chooses it in $\mathcal{F}(S, b - 1, a - 1)$. Children are selected independently.

LEMMA 4 (SUFFICIENT STATE FOR A SUFFIX TREE). *The recurrence gives exactly the nondominated immediate-preparation tree bounds. Here b counts the remaining allowed bad branches; it need not equal actual unobserved writes remaining.*

PROOF. A path with d bad branches needs at least d distinct invalidating writes. Conversely, exactly one fresh own-key write at each bad gate and no others realizes that path. Hence worst costs with at most b writes equal maxima over paths with at most b bad branches. Inert writes or multiple writes at one gate add no new path.

Induct on $|S| + a$. At $S = \emptyset$ there is no cost. At $b = 0$, prepare/cached completes every ready job with no duplicate or protected work, dominating other suffixes. Otherwise a fresh node has one child on R . A comparison has a matching child with allowance b and a bad child with allowance $b - 1$; a cheap failure additionally consumes one of a retries and leaves S unchanged. Each child decreases the induction rank. Readiness depends only on S ; under immediate preparation, costs depend only on the selected job/mode and outcome. These facts yield exactly the displayed charges.

The two children describe mutually exclusive observations. The controller may choose either continuation independently in advance; the adversary follows only one. It therefore receives no split budget across children. Taking a maximum separately in each resource is both necessary and sufficient for the two worst-case caps. Conversely every tree has precisely this root/child decomposition. Monotonicity of each charge/max expression allows a dominated child to be replaced by a dominating one. Emitted policies retain witness nodes; original histories with the same (S, b, a) need not be equivalent. Theorem 7 supplies full-program completeness. $\square$

There are at most $(r + 1)(\min(B, n + r) + 1)2^n$ states, with possibly exponential frontier sizes. The independent-job and enough-retry DAG formulas identify polynomial regions; a general fixed-order theorem is not claimed.

The least unaware work curve for general DAGs remains open.

5.2 What the Cached Comparison Reveals

Consider an erased observation contract. Every invocation returns the same immutable per-job object z_i , which completion publishes over its live entry. The cached primitive's *entire visible transition* exposes only that output and completion: no compared input, Boolean, comparison-dependent record selection or callback-local side effect survives. Kernel internals, work counters, allocation IDs and clocks are hidden. Free outside inspections and old snapshots remain allowed. Constant output alone is insufficient if the callback saves the current input elsewhere.

PROPOSITION 8 (A STRICT OBSERVATION BOUNDARY). *For independent works $(8, 16)$, supplied $B = 1$ and $Q \leq 2$, the visible-comparison frontier is $\{(24, 24), (32, 16)\}$; erasure changes it to $\{(24, 24), (40, 16)\}$.*

PROOF. All-fresh attains $(24, 24)$. For $(32, 16)$, cache the 8-unit job; on match complete the other fresh, and on mismatch prepare/cache it. The latter branch certifies the sole write. Theorem 4 gives the visible frontier.

For erasure, follow any admitted program's no-write run. Cheap would permit an extra failed call, so both completions are guarded. Partition them into matching cached C and fresh/already-stale cached F . If F is nonempty, either both bodies are already protected or a single own-gate write at the remaining matching cached job protects its body too. Erasure restores the same visible state after that completion, preserving subsequent inspections, choices and protected F bodies. Thus worst $L \geq 24$ is unavoidable.

If $L < 24$, both jobs must instead be matching cached on that path. One fresh write at the 16-unit job forces $W \geq 24 + 16 = 40$ and $L \geq 16$, by Lemma 2. Immediate prepare/cache for both attains $(40, 16)$: at most one comparison can mismatch. These cases exhaust programs and prove the erased frontier. $\square$

A complete erased-observation frontier for general positive-retry DAGs remains open.

6 Actual External Writes in Java

Java 17’s map API and the pinned implementation supply the per-key synchronization premises below [21–23]. The transfer is conditional; finite executions are not a JDK/JMM proof.

Use Section 2’s whole kernels, captured inputs, saved parents and prior store, with one controller and permanent distinct keys. Values are deeply immutable, non-null and identity-equal; replacement may call equals. Key methods are stable, side-effect-free and normally terminate. Kernels make no recursive/other-key updates. Writers preserve shape/milestones and admit references fresh to all preceding target records/snapshots. Direct locking and retained guards are excluded. Selected operations terminate; waiting is uncharged.

Cheap wraps replace or validation-only compute, exposing success/failure; cached returns its saved output/comparison Boolean; fresh always computes. Observations retain reference equality, payloads, wrapper results, saved parents and selector state. Raw cheap-mismatch values, numeric IDs, clocks and diagnostics are hidden. Erasure requires Section 5.2’s constant per-job output and full-history contract; hiding only the flag can leak through result identity.

Correspondence. An injective map η preserves references in the live map, available records, saved results, pending writes and ghost computation history. Only actually available handles enter $\mathcal{R}$; a discarded ghost record cannot be selected. Set $\mathcal{M}(i) = \eta(\text{done}[i])$ from the saved output, and map a record to $(i, \eta(s), \mathcal{M}|_{\text{Pred}(i)}, \eta(z))$. Let Δ queue committed external assignments deferred during normalization. At boundaries,

$$\text{decode}_\eta(H.\text{map}) = \text{apply}(\Delta, x), \quad b_H = b_x + |\Delta|.$$

Here b counts committed external publications, and Δ replays assignments rather than callbacks. Retrieval returns a per-key value at a point during its call; replacement checks/stores atomically. compute invokes remapping once, excluding same-key publications through its store. Wrappers save its result even if the map changes before return. These API/source premises cover list/tree bins.

THEOREM 9 (CONDITIONAL RESOURCE TRANSFER FOR ACTUAL WRITES). *Under these premises and each result’s observation contract, the call/body-work results of Sections 3–5 hold for actual external writes, with separate or shared bins. Completed-operation traces preserve declared results and Q, W, L , operation kinds and cached-mismatch labels; hence they also preserve Q' of Section 4.4.*

PROOF. *Upper-bound projection.* Group each completed *unprotected* kernel with its captured operands, returned record and full charge as Prepare. Defer writes since capture in Δ , flushing them before the next operation. The record/charge is placed after the source kernel returns, not at an unfinished prefix. For a standalone inspection, flush commits through its read point before that read; queue later commits, preserving the saved return. A protected body stays inside its fresh/cached operation. Other-key commits during that operation may precede its abstract effect: the kernel has no live other-key reads and saved parents are immutable. Same-key writes are excluded until publication.

Post-publication/pre-return writes enter Δ ; completion saves the prepared output for successful cheap calls and the returned output for callbacks, never a map reread. Flush any terminal queue as a zero-foreground-cost write suffix. Resize/tree conversion copies references; a concurrent publication remains a write. This completed-trace normalization preserves declared foreground observations and all charges without increasing writes. It is not an instruction-level simulation or equivalence of writer diagnostics. Explicit upper policies contain bounded finite segments, so selected-operation progress gives completion and transfers their bounds.

Concrete lower witnesses. After the controller fixes its job/mode and prior store, suspend it before API entry. With no foreground guard held, a terminating own-key writer installs a fresh reference and finishes, even with colliding bins. It invalidates every retained target record. Fix these concrete writer results at the restricted histories used in the lower proofs. The target adversaries preserve their unconditional write caps; Theorem 7’s concrete-history extraction and global distinct charges apply to general programs, including early preparation. For a path needing d writes, its witness stops off-path and after d writes on every history. These particular realizable witnesses suffice; arbitrary abstract environments need not all be implemented in Java. Saved parents preserve readiness, and callback Booleans provide the permitted branches.

The source-to-gate projection preserves operation-kind and cached-mismatch ghost labels, giving $Q' = Q + \# \text{cached mismatches}$. This is separate from program-to-tree domination. The controller may count its visible calls without observing hidden resource labels. $\square$

Earlier kernels perform $8w_i$ iterations; newer wrappers use w_i . Waiting, allocation, traversal and writer work are uncharged. Source control points, transitions and the full relation are archived [26]. Finite tests establish neither API premises nor concurrent-resize correctness.

7 Charging Entry Work Changes Joint-Cap Complexity

Body-work accounting isolates kernel recomputation. A protected callback also enters a guard and may compare a record. Give these actions nonnegative supplied costs g_i and c_i , respectively; fresh completion pays only g_i , and cheap calls have zero added charge. Define

$$W_t = W + \sum_{\text{guarded}} g_i + \sum_{\text{cached}} c_i, \quad P_t = L + \sum_{\text{guarded}} g_i + \sum_{\text{cached}} c_i.$$

These mathematical prices are not calibrated native times. The guard ends on return; charges add no observations, skipped comparisons or cross-job amortization. Cached mismatch still completes in one call. Throughout this section, $B = 1$ is supplied and $Q \leq n$. A protected call fee $d_i \geq 0$ common to all three modes adds $\sum_i d_i$ to both coordinates: every job completes once and $Q = n$. The formulas omit this policy-independent constant.

For a topological order π and cached subset C , set $F = J \setminus C$, $h_i = g_i + c_i \mathbf{1}[i \in C]$, and $H = \sum_i h_i$. Let H_i sum h_j through i and F_i sum the works of F jobs strictly before i . Empty maxima are 0.

THEOREM 10 (COMPLETE ONE-WRITE REPRESENTATION). *The complete frontier is the nondominated set over all (π, C) of*

$$\begin{aligned} W(\pi, C) &= \Omega + \max\{H, \max_{i \in C} (w_i + H_i)\}, \\ P(\pi, C) &= \max\{H + \Omega(F), \max_{i \in C} (w_i + H_i + F_i)\}. \end{aligned}$$

An order, a mode bit per job and a first-mismatch flag specify an attaining policy.

PROOF. Follow π , using fresh on F and immediate prepare/cache on C until the first mismatch; then prepare/cheap-complete the suffix. A mismatch of this just-prepared record certifies the sole write, so all suffix calls succeed. Without a mismatch, costs are $(\Omega + H, H + \Omega(F))$. A first mismatch at i gives $(\Omega + w_i + H_i, w_i + H_i + F_i)$. No writes and one selected own-gate write realize these cases.

For an arbitrary admitted program, fix a no-write execution. Cheap is impossible: a fresh gate write would add a failure to n mandatory completions. Guarded completions induce π ; take matching cached jobs as C and fresh/already-stale cached jobs as F . Dropping stale cached comparison/preparation charges only weakens the lower bound. This execution costs at least $(\Omega + H, H + \Omega(F))$.

For each $i \in C$, preserve the prefix, write one fresh own-key identity at its gate, then stop. This invalidates all retained target records. Lemma 2 forces body work at least $\Omega + w_i$, including paid early preparation. The incurred guarded/comparison prefix is at least H_i ; protected bodies include $F_i + w_i$. Thus this alternative costs at least $(\Omega + w_i + H_i, w_i + H_i + F_i)$. Each environment has an unconditional one-write cap. The program's separate suprema dominate every witness and hence the displayed pair. The attaining policy proves completeness, including input-dependent choices and retained records. $\square$

The supplement gives independent-job subset representations, a linked SUBSET SUM reduction [4], pseudopolynomial weighted-deadline DP [2, 3], and polynomial common-guard frontiers [26]. Below, precedence changes the answer even with unit guards.

At $c_i = 0$, protected cost alone is polynomial: all-cached reduces Theorem 10 to Lawler's precedence problem with processing g_i , tail w_i [37]. Hardness concerns *joint* entry-charged caps.

7.1 Unit Entry Costs Make Precedence Hard

With $g_i = c_i = 0$, an optimal policy under $W \leq \Omega + M$, $M \geq 0$, can cache every job with $w_i \leq M$: in any fixed topological order, each such replacement introduces a mismatch term bounded by the old no-write protected load and decreases later fresh-body prefixes. Set $C = \{i : w_i \leq M\}$, $F = J \setminus C$, processing $p_i = w_i \mathbf{1}[i \in F]$ and tail $q_i = w_i \mathbf{1}[i \in C]$. Minimizing protection is exactly Lawler's objective $\max_i (C_i^p + q_i)$ [37], where C_i^p is processing completion time, including zero processing times. Its inherited rule at the at most $n + 1$ thresholds $M \in \{0\} \cup \{w_i\}$, followed by pruning, gives the complete frontier in $O(n^3)$ operations on any DAG. A cap below Ω is infeasible.

THEOREM 11 (UNIT-GUARD PRECEDENCE BOUNDARY). *With $c_i = 0$ and all $g_i = 1$, joint-cap decision is strongly NP-complete already on height-two DAGs with at most two predecessors per job and $w_i \in \{1, n + 1\}$. Nevertheless, every DAG with a common guard price g has at most $n^2 + 1$ distinct frontier points.*

PROOF. Membership follows from Theorem 10. We adapt Garey–Johnson's incidence/prefix construction [1, Theorem 2]. Take a simple graph and $k \geq 3$, with $K = \binom{k}{2}$; settle too few vertices/edges separately. Add an isolated edge and two vertices, preserving k -cliques and giving augmented counts $v \geq k + 2$, $m \geq K + 1$. Create v vertex jobs of work 1 and m edge jobs of work $H = n + 1$, $n = v + m$, each following its endpoints. Set

$$D = k + K, \quad M = D + H, \quad K_W = \Omega + M, \quad K_P = n + (m - K)H.$$

Then $n > D$ and $K_P > M > n$; the extra edge supplies room for mismatch cost.

Given a clique, order its vertices, its edges, other vertices, then other edges. Cache all vertices and the clique edges; finish other edges fresh. Every cached prefix has $F_i = 0$ and $\text{pos}(i) + w_i \leq M$. Theorem 10 gives $W \leq K_W$ and $P = K_P$.

Conversely, extract (π, C) from one fixed no-write execution of any feasible program. Its protected load $n + \Omega(F) \leq K_P$ forces at least K cached edge jobs. For each, a separate own-gate one-write witness on that same prefix requires $\text{pos}_\pi(i) + H \leq M$, hence position at most D . The first D jobs therefore contain at least K edges and at most k vertices, including every endpoint. A simple graph on at most k vertices with at least $\binom{k}{2}$ edges is a k -clique. Padding creates none. All numerical parameters are $O(n^2)$, giving strong hardness.

For common g , Theorem 10 restricts the first coordinate to

$$\{\Omega + ng\} \cup \{\Omega + w_i + jg : 1 \leq i, j \leq n\}.$$

Only the least second coordinate for each first coordinate can be nondominated, proving the cardinality bound. $\square$

The supplementary heterogeneous-guard construction attains $2^n - 1$ independent-job frontier points [26].

8 Formal Corroboration and Operational Evaluation

Inputs are fixed before execution; post-outcome reanalyses and all failures/corrections remain explicit and archived.

Executions use Apple M2/24 GiB, Python 3.10.9/3.12.14 and OpenJDK 17.0.19; the pinned map source was inspected, not rebuilt. Hashes bind code, inputs and traces. Codex drafted proofs, implementations, input generators, checker/analysis scripts, figures and artifact packaging, and ran experiments. Claude supplied technical critiques and fixed-manuscript reviews. These are author-side AI-assisted checks; no human study or independent certification is claimed.

An input fixes graph/works/prices; a root adds budget/retries. Paths are complete branch sequences, executions native invocations, and pairs compare policies on one input. Coordinates add evaluation budgets. The studies overlap.

8.1 RQ1: Bounded Corroboration of the Proofs

Table 3. Bounded checks of the retained claims. Populations overlap; finite agreement does not prove all-program completeness.

Obligation	Evidence and comparison
General call/ L values	Direct minimax versus formulas: 265,629 agreeing roots from 5,421 DAG/weight inputs.
General retry frontier	Immediate and retained-preparation finite models: 96,795 agreeing cases; $n \leq 4$, works $\{1, 2, 4\}$, $r = 0, 1, 2$.
Polynomial formulas	Retrospective: 1,998 independent and 32,526 enough-retry DAG cases (720 overlap). Prospective: 13,850 agreeing roots and 540,230 independently interpreted paths.
Unknown-budget curves	647 fixed independent roots and 34,814 interpreted paths; exact least curves agree, zero mismatches. Earlier 713 general roots remain.
DAG/observation boundaries	36 scope roots, zero mismatches; four erasure roots, 8,672 trees and 102,122 paths. All 8,668 protection violations retained.
Charged representation	14,696 inputs; all order/mask and interpreted-path comparisons agree.
Free-comparison boundary	764 independent and 836 common-guard inputs; all policy/cap comparisons agree.
Unit-guard precedence	3,208 source cases; 1,399 reductions (728 yes/671 no); 675 common-guard DAG cases. All agree.
Changed call unit	13,176 supplied roots from 1,098 inputs and 222 unaware curves from 74 inputs; all agree with Proposition 6.

The prospective general-retry study uses 461 multisets, $n \leq 6$, works $\{1, 2, 4, 7, 11\}$, $r \leq 3$, $B \leq n+r$. All 9,240 first-excluded-budget controls violate L , preserving Q ; Boolean-value cases are archived.

Both wrappers cover colliding/separate and tree bins, and canonical/allocating outputs. All 6,112 supplied and 10,192 unaware pairs agree. The unaware selector sees no B , flag or diagnostics. Twelve corruptions reject; four of 52 flag-ignoring controls violate L . All outcomes and tightness witnesses remain archived.

Unit-guard checks cover all graph/ k cases on 3–5 vertices, including 1,809 trivial negatives. Constructive paths respect both caps; policy maxima attain them separately. Fee recurrences

translate frontiers; three alterations reject. Supplementary independent-price checks retain 420 targets (339 yes/81 no), 836 reused greedy inputs and seven rejected alterations.

8.2 RQ2: A Native Traversal and Its Guard Boundary

Linux v6.12 `dentry_path_raw` first walks mutable `dentry` parents/names under RCU, validates `rename_lock`'s sequence, and on failure rebuilds under its reader-exclusive lock [10]. Pathname lock contention motivated the upstream optimistic fallback [12]. The related `is_subdir` comment explicitly connects one lockless try with fallback for progress [11]. In this pathname loop, the caller sets the marker to 1 after failure; the primitive does not automatically count retries. Our KUnit-only variant prepares a second traversal with a *new even* sequence, then acquires the lock and validates that saved sequence: reuse on match, reset/rebuild on mismatch. Every traversed component must be initialized, with no concurrent initialization; a valid reference alone is insufficient. Initial `dentry/name` publication must precede parallel reading; subsequent relevant changes must participate in the global sequence, without wrap. This covers the initialized `d_exchange` fixtures, not every filesystem: `d_mark_tmpfile` changes names without that sequence during initialization [11]. Mount traversal and `getcwd` are excluded.

Assume a private valid buffer with $1 \leq \text{capacity} \leq \text{INT_MAX}$, nonnegative observed lengths fitting `int`, valid `dentry/RCU` lifetimes, and successful stable-name copies. Let $C = \text{capacity} - 1$ after NUL. In the original factored loop, attempted names v , requested copy bytes c , fault-fill bytes f and successful slashes s satisfy $v \leq s + 1$, $f \leq c$ and $c + s \leq C$. Thus body events $T = 1 + v + c + f + s \leq 2C + 2$. For first/second validation failures X, Y , fresh even capture gives $X + XY \leq \min(B, 2)$ for actual global writer sections, including unrelated renames. Count one Q unit per lockless validation or guarded validation/completion block, not per native API call. The candidate has $Q \leq 2$, $T_{\text{total}} \leq (1 + \min(B, 2))(2C + 2)$ and $L_T = 0$ for $B \leq 1$. This is a single-job upper refinement: private-buffer return does not instantiate the full publication/observation contract. Guard/RCU instructions, waiting and postambles are separate; the capacity-4096 ceiling 8192 is loose on short paths.

ARM64 Linux runs in two-CPU QEMU with lockdep and atomic-sleep checking. A pinned native `lock_rename/d_exchange` writer uses endpoint gates outside RCU and a nonsleeping mid-parent gate. The fixed matrix has 1,920 rows (948 paths/972 expected overflows) and four detected defects. A fixture oracle checks strings, offsets, canaries and post-return persistence against quiescent upstream. Outputs/body records agree without lock/RCU diagnostics. Even nonwrapping sequence brackets confirm designated writer counts and all prior output/body rows. Returning unvalidated overflow, failing to restore the buffer or ignoring final validation breaks output. Reusing the first sequence preserves output but violates the protection bound at $B = 1$. No copy-fault/allocation failure is injected; native-time performance is unmeasured.

Table 4. Source event counts at the scripted gates. Initial path is `/driver/cache/entry/leaf`; first exchange moves it to `/alternate`, second restores it. G counts reader-exclusive acquisitions.

Policy	One write				Two writes			
	T	L_T	Q	G	T	L_T	Q	G
Upstream	41	12	2	1	58	29	2	1
Cached	41	0	2	1	70	29	2	1
Two cheap, then locked	41	0	2	0	70	29	3	1

Count a body attempt whenever reconstruction starts from the original `dentry/buffer`, including an overflow exit. Upstream has $1 + X \leq 2$ attempts; cached has $1 + X + XY \leq 3$. The latter preserves

bounded completion under terminating primitives but relaxes upstream’s two-attempt bound. It is not a proof of faster completion. All 384 cached/two-cheap pairs agree on output and body vectors; 138 first-invalid/second-valid pairs avoid the guard, while 144 two-failure pairs need another logical block. The cap comparison depends on its unit. One public `dentry_path_raw` invocation remains one under every policy. Counting six named primitives—RCU enter/exit, sequence begin/retry, and exclusive lock/unlock—gives $4R + 2G$, where R counts outside traversals. The cached/two-cheap counts are 4/4, 10/8, 10/10 for zero, one and two observed failures: both universal maxima are 10, despite logical caps 2/3. These are source call counts, excluding spinning and underlying instructions. No caller-required $Q = 2$ SLA was found. A stronger native name-snapshot comparator copies inline names or retains external-name references under protection, then materializes outside. Its allocation, dentry locks, capture steps and refcounts are separate from L_T ; five 64-slot overflows retain the discarded capture work before protected fallback. Its body-only zero is not zero total protected work or a lower bound for larger workspaces. All adverse results and the first 1,536-row matrix remain; the populations overlap.

8.3 RQ3: Preservation and Cost Limits in Roslyn

Roslyn’s `Workspace.cs` at 6c4a46a3 transforms an immutable solution outside a semaphore, validates identity inside and retries [13, 14]. Our `SourceText`-only patch switches after r shared failures to fresh (two-mode) or cached completion (three-mode). A stronger rebase reuses guarded document contents but still requires a protected merge. Lazy work, allocation, version generation and cache-dependent merges prevent fixed-work transfer. No upstream numerical cap or SLA was found.

All 236 authored cases agree; 12 corruptions reject. Full event/publication/snapshot records are archived. In 96 compiler processes, 2,556 project projections match declared expectations for references, parse options and linked text under cold/warm caches; eight corruptions reject. The original same-text deadlock and four missing-document timeouts remain archived; corrected patches pass 14 error cases and reentry controls. These selected checks use Roslyn on both sides. Custom loaders, changed-text reentry and general concurrent equivalence remain outside scope.

Arrival tests compare shared/per-update caps, two/three modes and both baselines. Eight balanced blocks serve 32 updates and 64 FIFO writers each at 0, 10, 100, 1000 μ s. All 2,560 measured and 512 warm-up units pass source/count checks; 12 corruptions reject. At $r = 2$, local/shared caps allow 96/34 calls, precluding equal-cap comparison.

Shared three/two foreground-time ratios at 0/10 μ s are 1.72 [1.40,2.11] and 1.58 [1.17,2.16], favoring two modes. Slower-arrival and all writer-response intervals include 1. Geometric means use eight paired block ratios and descriptive 95% intervals from 10,000 fixed block-bootstrap samples without multiplicity correction. Baseline ratios 0.83–1.08 all include 1; instrumentation overhead is unidentified. Authored arrivals, eight blocks and one host limit inference; the supplement retains all mixed results.

9 Related Work and Research Lineage

Fallback and contention. Protected reexecution [9, Section 3.3], bounded lock-elision/RPC retries [29, 30, 34, 35], irrevocable prefix preservation [36], contention management [7, 27, 28] and switching/tuning [8, 44, 45] are established. HTM aborts need not consume writes [31], and RPC can fail. Their progress/time objectives differ from persistent-job W/L caps.

Synthesis. Černý et al. [18] synthesize retries, guards and caches for weighted limit-average safety (PSPACE by erratum); later work optimizes protection/profile costs [19] or preemption-safe

locks [20]. We inherit global-budget/information-state minimax DP [15, 24, 32], backward induction and Pareto composition; our results supply all-program domination and exact polynomial regions.

Scheduling with testing. At $r = 0$, fresh completion is untested execution of cost $u_i = w_i$; cached completion tests for $t_i = w_i$, then executes for $p_i \in \{0, w_i\}$. Thus W counts testing plus execution, and L execution alone. A deterministic tree’s mismatch set can be fixed as hidden bad-job labels, and own-gate writes realize those labels. Dynamic writing alone does not distinguish this canonical game.

Dürr et al. [38] introduce adversarial testing; Buld and Schulz [41] and Albers and Eckl [42] allow unequal test times/upper bounds, for competitive time objectives. Damerius et al. [39] separate testing expenditure and execution with a *policy-side* budget. Our B limits adverse outcomes; preparations remain allowed at $B = 0$. Dufossé et al. [40] study minimax testing trees with durations revealed even for untested execution; polynomial adaptive optimality assumes their two-phase conjecture. Albers and Janke [43] bound failing jobs for irrevocable parallel-machine assignment. Our specialization returns causal completion policies for separate absolute W/L caps and retry slack. Separate worst testing/execution bounds do not determine worst total work: in the 8/16 example, the adaptive policy and unconditional prepare/cache-both have $(\sup T, \sup L) = (24, 16)$ but $\sup(T + L) = 32$ and 40. Here T is unprotected preparation work. This explains an objective difference, not newness of adaptive testing; the structural frontier and domination proofs carry that claim. The informed comparator knows only B .

Precedence and reuse. Section 7 embeds inherited precedence [37], weighted-deadline [2] and incidence-hardness [1] results. Universal restart [46] concerns expected Las Vegas time. Build and incremental systems [5, 6, 33] may reuse partial work outside our whole-kernel interface.

10 Limitations and Threats to Validity

Section 2 requires deterministic, fully charged whole kernels and excludes retained guards. Least unaware work curves for general DAGs remain open.

Supplied B is a premise. Native costs are uncalibrated; instrumentation, caches and runtime compilation limit timings. The archive retains six scaling timeouts, retained-guard/simple-rule counterexamples, reversed disjoint-bin timings, scalar gains in only 2/96 cells and successful cost fits in 2/8. All-budget compilation can cost more than requested-value computation. The archived Valkey study compares a reader-repaired digest against cooperating writer-maintained materialization: all 16 cells favor writer maintenance in time/transfer, with mixed maximum command times. These larger interfaces delimit our claims. Finite/AI-assisted checks are neither mechanical proofs nor independent reproduction; human benefit is unmeasured.

11 Conclusion

We characterize joint total/protected-work guarantees through exact independent-job frontiers, least unaware work curves and all-program finite-tree domination. Observation and accounting change these answers. The conditional Java transfer, Linux’s two-versus-three body attempts and mixed Roslyn timings show which additional premises matter when choosing a source implementation.

Data Availability

Code, fixed inputs, all outcomes/corrections, proofs, replay commands and licenses are archived [26]. References [25, 26] are public snapshots of one author artifact lineage. The latter extends the earlier retry study with work-frontier, observation, source and charged-resource developments; neither is an independent novelty baseline.

References

- [1] Michael R. Garey and David S. Johnson. 1976. Scheduling Tasks with Nonuniform Deadlines on Two Processors. *Journal of the ACM* 23, 3, 461–467. Theorem 2 concerns one processor. <https://doi.org/10.1145/321958.321967>.
- [2] Eugene L. Lawler and James M. Moore. 1969. A Functional Equation and its Application to Resource Allocation and Sequencing Problems. *Management Science* 16, 1, 77–84. <https://doi.org/10.1287/mnsc.16.1.77>.
- [3] Karl Bringmann, Nick Fischer, Danny Hermelin, Dvir Shabtay, and Philip Wellnitz. 2020. Faster Minimization of Tardy Processing Time on a Single Machine. In *ICALP*, Article 19, 19:1–19:12. Theorem 1.1 restates Lawler–Moore’s weighted-tardiness bound. <https://doi.org/10.4230/LIPICs.ICALP.2020.19>.
- [4] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. Reprint: <https://www.cs.umd.edu/~gasarch/BLOGPAPERS/Karp.pdf>.
- [5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [6] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/popl09.pdf>.
- [7] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [8] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [9] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [10] Linux contributors. 2024. Linux v6.12, fs/d_path.c and fs/hostfs/hostfs_kern.c, commit adc218676eef25575469234709c2d87185ca223a. Accessed September 11, 2026. https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/d_path.c.
- [11] Linux contributors. 2024. Linux v6.12, fs/dcache.c: name snapshots, d_exchange, and tmpfile initialization. Accessed September 11, 2026. <https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/dcache.c>.
- [12] Waiman Long. 2013. dcache: Translating dentry into pathname without taking rename_lock. Linux commit 232d2d60aa5469bb097f55728f65146bd49c1d25. <https://github.com/torvalds/linux/commit/232d2d60aa5469bb097f55728f65146bd49c1d25>.
- [13] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [14] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [15] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [16] Avi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [17] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [18] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [19] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [20] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [21] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [22] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [23] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.

- [24] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [25] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [26] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Total-work, comparison-free and unit-guard precedence, and call-accounting supplement, commit f626c4c. <https://github.com/research-artifact-archive/research-artifact/tree/f626c4c412f422b975e5c21d7251090e4a5b9fec>.
- [27] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
- [28] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint arXiv:1804.00947v1*. <https://arxiv.org/abs/1804.00947>.
- [29] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651. `elision-lock.c` and `elision-conf.c`. <https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfaca0d86cfb66/sysdeps/unix/sysv/linux/x86>. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.
- [30] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cafa0defb956e13a2c/src/hotspot/cpu/x86>.
- [31] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
- [32] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint arXiv:2301.05089v2*. <https://arxiv.org/abs/2301.05089>.
- [33] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adaption: Composable, Demand-Driven Incremental Computation. In *PLDI*. <https://doi.org/10.1145/2594291.2594324>. Author version: <https://www.cs.tufts.edu/~jfofster/papers/pldi14.pdf>.
- [34] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. <https://sre.google/sre-book/addressing-cascading-failures/>.
- [35] Envoy contributors. n.d. CircuitBreakers.Thresholds.RetryBudget. Accessed September 10, 2026. https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html.
- [36] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. <https://doi.org/10.1145/1378533.1378584>.
- [37] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>. Author technical report ERL-M327 (1971): <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1971/Archive/ERL-m-327.pdf>.
- [38] Christoph Dürr, Thomas Erlebach, Nicole Megow, and Julie Meißner. 2020. An Adversarial Model for Scheduling with Testing. Author preprint arXiv:1709.02592v3. <https://arxiv.org/abs/1709.02592v3>.
- [39] Christoph Damerius, Peter Kling, Minming Li, Chenyang Xu, and Ruilong Zhang. 2023. Scheduling with a Limited Testing Budget: Tight Results for the Offline and Oblivious Settings. Author preprint arXiv:2306.15597v1. <https://arxiv.org/abs/2306.15597v1>.
- [40] Fanny Dufossé, Christoph Dürr, Noël Nadal, Denis Trystram, and Óscar C. Vázquez. 2021. Scheduling with a Processing Time Oracle. Author preprint arXiv:2005.03394v3. <https://arxiv.org/abs/2005.03394v3>.
- [41] Felix Buld and Andreas S. Schulz. 2026. Scheduling with Testing: Competitive Algorithms for Minimizing the Total Weighted Completion Time in the Adversarial Model. Author preprint arXiv:2606.25166v1. <https://arxiv.org/abs/2606.25166v1>.
- [42] Susanne Albers and Alexander Eckl. 2021. Explorable Uncertainty in Scheduling with Non-Uniform Testing Times. Author preprint arXiv:2009.13316v2; first version 2020. <https://arxiv.org/abs/2009.13316v2>.
- [43] Susanne Albers and Maximilian Janke. 2021. Online Makespan Minimization with Budgeted Uncertainty. In *WADS*. https://doi.org/10.1007/978-3-030-83508-8_4. Author version: <https://maximilian-janke.de/data/BudgetedUncertainty.pdf>.
- [44] Beng-Hong Lim and Anant Agarwal. 1994. Reactive Synchronization Algorithms for Multiprocessors. In *ASPLOS VI*, 25–35. Author version: <https://groups.csail.mit.edu/cag/pub/papers/pdf/reactive.pdf>.
- [45] Takayuki Usui, Reimer Behrendts, Jacob Evans, and Yannis Smaragdakis. 2010. Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency. *Journal of Parallel and Distributed Computing*. <https://doi.org/10.1016/j.jpdc.2010.02.006>. Author version: <https://people.cs.umass.edu/~yannis/adaptive-jpdc.pdf>.
- [46] Michael Luby, Alistair Sinclair, and David Zuckerman. 1993. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, 4, 173–180. Author version: <https://www.cs.utexas.edu/~diz/pubs/speedup.pdf>.